

## Lab # 10

### Implementation of Aliases & Joins using Query Language

#### Objective:

Student will learn about to do the following:

- Alias in SQL
- Join in SQL

#### Software Tools:

- MySQL Workbench

#### Theory:

SQL aliases are used to give a table, or a column in a table, a temporary name. It is often used to make column names more readable. It only exists for the duration of the query.

Aliases can be useful when:

- There are more than one table involved in a query
- Functions are used in the query
- Column names are big or not very readable
- Two or more columns are combined together

#### **Alias Column Syntax**

SELECT *column\_name* AS *alias\_name*

FROM *table\_name*;

#### **Example**

SELECT CustomerID AS ID, CustomerName AS Name FROM Customers;

#### **Alias Table Syntax**

SELECT *column\_name(s)*

FROM *table\_name* AS *alias\_name*;

#### **Example**

SELECT o.OrderID, o.OrderDate, c.CustomerName

FROM Customers AS c, Orders AS o

WHERE c.CustomerName="Around the Horn" AND c.CustomerID=o.CustomerID;

## SQL JOIN

SQL joins are used to query data from two or more tables, based on a relationship between certain columns in these tables. The JOIN keyword is used for this purpose. Tables in a database are often related to each other with keys. A primary key is a column (or a combination of columns) with a unique value for each row. Each primary key value must be unique within the table. The purpose is to bind data together, across tables, without repeating all of the data in every table.

Look at the "Persons" table:

| P_Id | LastName  | FirstName | Address      | City      |
|------|-----------|-----------|--------------|-----------|
| 1    | Hansen    | Ola       | Timoteivn 10 | Sandnes   |
| 2    | Svendson  | Tove      | Borgvn 23    | Sandnes   |
| 3    | Pettersen | Kari      | Storgt 20    | Stavanger |

Note that the "P\_Id" column is the primary key in the "Persons" table. This means that no two rows can have the same P\_Id. The P\_Id distinguishes two persons even if they have the same name.

Next, we have the "Orders" table:

| O_Id | OrderNo | P_Id |
|------|---------|------|
| 1    | 77895   | 3    |
| 2    | 44678   | 3    |
| 3    | 22456   | 1    |
| 4    | 24562   | 1    |
| 5    | 34764   |      |

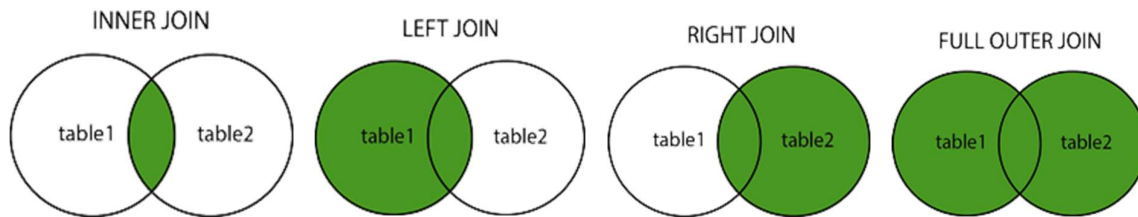
Note that the "O\_Id" column is the primary key in the "Orders" table and that the "P\_Id" column refers to the persons in the "Persons" table without using their names.

Notice that the relationship between the two tables above is the "P\_Id" column.

## Different SQL JOINS

Before we continue with examples, we will list the types of JOIN you can use, and the differences between them.

- **JOIN/ INNER JOIN/ SIMPLE JOIN:** Return rows when there is at least one match in both tables
- **LEFT JOIN:** Return all rows from the left table, even if there are no matches in the right table
- **RIGHT JOIN:** Return all rows from the right table, even if there are no matches in the left table
- **FULL JOIN:** Return rows when there is a match in one of the tables.



## SQL INNER JOIN Keyword

The INNER JOIN keyword return rows when there is at least one match in both tables.

## SQL INNER JOIN Syntax

```
SELECT column_name(s)
FROM table_name1
INNER JOIN table_name2
ON table_name1.column_name=table_name2.column_name
```

**PS:** INNER JOIN is the same as JOIN.

## SQL INNER JOIN Example

The "Persons" table:

| P_Id | LastName  | FirstName | Address      | City      |
|------|-----------|-----------|--------------|-----------|
| 1    | Hansen    | Ola       | Timoteivn 10 | Sandnes   |
| 2    | Svendson  | Tove      | Borgvn 23    | Sandnes   |
| 3    | Pettersen | Kari      | Storgt 20    | Stavanger |

The "Orders" table:

| O_Id | OrderNo | P_Id |
|------|---------|------|
| 1    | 77895   | 3    |
| 2    | 44678   | 3    |
| 3    | 22456   | 1    |
| 4    | 24562   | 1    |
| 5    | 34764   |      |

Now we want to list all the persons with any orders.

We use the following SELECT statement:

```
SELECT Persons.LastName, Persons.FirstName, Orders.OrderNo
FROM Persons
INNER JOIN Orders
ON Persons.P_Id=Orders.P_Id
ORDER BY Persons.LastName
```

The result-set will look like this:

| LastName  | FirstName | OrderNo |
|-----------|-----------|---------|
| Hansen    | Ola       | 22456   |
| Hansen    | Ola       | 24562   |
| Pettersen | Kari      | 77895   |
| Pettersen | Kari      | 44678   |

The INNER JOIN keyword return rows when there is at least one match in both tables. If there are rows in "Persons" that do not have matches in "Orders", those rows will NOT be listed

## SQL LEFT JOIN Keyword

The LEFT JOIN keyword returns all rows from the left table (table\_name1), even if there are no matches in the right table (table\_name2).

### SQL LEFT JOIN Syntax

```
SELECT column_name(s)
FROM table_name1
LEFT JOIN table_name2
ON table_name1.column_name=table_name2.column_name
```

**PS:** In some databases LEFT JOIN is called LEFT OUTER JOIN.

### SQL LEFT JOIN Example

The "Persons" table:

| P_Id | LastName  | FirstName | Address      | City      |
|------|-----------|-----------|--------------|-----------|
| 1    | Hansen    | Ola       | Timoteivn 10 | Sandnes   |
| 2    | Svendson  | Tove      | Borgvn 23    | Sandnes   |
| 3    | Pettersen | Kari      | Storgt 20    | Stavanger |

The "Orders" table:

| O_Id | OrderNo | P_Id |
|------|---------|------|
| 1    | 77895   | 3    |
| 2    | 44678   | 3    |
| 3    | 22456   | 1    |
| 4    | 24562   | 1    |
| 5    | 34764   |      |

Now we want to list all the persons and their orders - if any, from the tables above.

We use the following SELECT statement:

```
SELECT Persons.LastName, Persons.FirstName, Orders.OrderNo
FROM Persons
LEFT JOIN Orders
ON Persons.P_Id=Orders.P_Id
ORDER BY Persons.LastName
```

The result-set will look like this:

| LastName  | FirstName | OrderNo |
|-----------|-----------|---------|
| Hansen    | Ola       | 22456   |
| Hansen    | Ola       | 24562   |
| Pettersen | Kari      | 77895   |
| Pettersen | Kari      | 44678   |
| Svendson  | Tove      |         |

The LEFT JOIN keyword returns all the rows from the left table (Persons), even if there are no matches in the right table (Orders).

## SQL RIGHT JOIN Keyword

The RIGHT JOIN keyword Return all rows from the right table (table\_name2), even if there are no matches in the left table (table\_name1).

## SQL RIGHT JOIN Syntax

```
SELECT column_name(s)
FROM table_name1
RIGHT JOIN table_name2
ON table_name1.column_name=table_name2.column_name
```

**PS:** In some databases RIGHT JOIN is called RIGHT OUTER JOIN.

## SQL RIGHT JOIN Example

The "Persons" table:

| P_Id | LastName  | FirstName | Address      | City      |
|------|-----------|-----------|--------------|-----------|
| 1    | Hansen    | Ola       | Timoteivn 10 | Sandnes   |
| 2    | Svendson  | Tove      | Borgvn 23    | Sandnes   |
| 3    | Pettersen | Kari      | Storgt 20    | Stavanger |

The "Orders" table:

| O_Id | OrderNo | P_Id |
|------|---------|------|
| 1    | 77895   | 3    |
| 2    | 44678   | 3    |
| 3    | 22456   | 1    |
| 4    | 24562   | 1    |
| 5    | 34764   |      |

Now we want to list all the orders with containing persons - if any, from the tables above.

We use the following SELECT statement:

```
SELECT Persons.LastName, Persons.FirstName, Orders.OrderNo
FROM Persons
RIGHT JOIN Orders
ON Persons.P_Id=Orders.P_Id
ORDER BY Persons.LastName
```

The result-set will look like this:

| LastName  | FirstName | OrderNo |
|-----------|-----------|---------|
| Hansen    | Ola       | 22456   |
| Hansen    | Ola       | 24562   |
| Pettersen | Kari      | 77895   |
| Pettersen | Kari      | 44678   |
|           |           | 34764   |

The RIGHT JOIN keyword returns all the rows from the right table (Orders), even if there are no matches in the left table (Persons).

## SQL FULL JOIN Keyword

The FULL JOIN keyword return ALL rows even when there is no match / a match in one of the tables.

### SQL FULL JOIN Syntax

```
SELECT column_name(s)
FROM table_name1
FULL JOIN table_name2
ON table_name1.column_name=table_name2.column_name
```

### SQL FULL JOIN Example

The "Persons" table:

| P_Id | LastName  | FirstName | Address      | City      |
|------|-----------|-----------|--------------|-----------|
| 1    | Hansen    | Ola       | Timoteivn 10 | Sandnes   |
| 2    | Svendson  | Tove      | Borgvn 23    | Sandnes   |
| 3    | Pettersen | Kari      | Storgt 20    | Stavanger |

The "Orders" table:

| O_Id | OrderNo | P_Id |
|------|---------|------|
| 1    | 77895   | 3    |
| 2    | 44678   | 3    |
| 3    | 22456   | 1    |
| 4    | 24562   | 1    |
| 5    | 34764   |      |

Now we want to list all the persons and their orders, and all the orders with their persons.

We use the following SELECT statement:



```
SELECT Persons.LastName, Persons.FirstName, Orders.OrderNo
FROM Persons
FULL JOIN Orders
ON Persons.P_Id=Orders.P_Id
ORDER BY Persons.LastName
```

The result-set will look like this:

| LastName  | FirstName | OrderNo |
|-----------|-----------|---------|
| Hansen    | Ola       | 22456   |
| Hansen    | Ola       | 24562   |
| Pettersen | Kari      | 77895   |
| Pettersen | Kari      | 44678   |
| Svendson  | Tove      |         |
|           |           | 34764   |

The FULL JOIN keyword returns all the rows from the left table (Persons), and all the rows from the right table (Orders). If there are rows in "Persons" that do not have matches in "Orders", or if there are rows in "Orders" that do not have matches in "Persons", those rows will be listed as well.

## LAB TASKS:

1): Consider the following two tables:

a) Persons table is as follow:

| P_Id | LastName  | FirstName | Address      | City      |
|------|-----------|-----------|--------------|-----------|
| 1    | Hansen    | Ola       | Timoteivn 10 | Sandnes   |
| 2    | Svendson  | Tove      | Borgvn 23    | Sandnes   |
| 3    | Pettersen | Kari      | Storgt 20    | Stavanger |

b) ORDER table is as follow:

| O_Id | OrderDate  | OrderPrice | Per_id |
|------|------------|------------|--------|
| 1    | 2008/11/12 | 1000       | 3      |
| 2    | 2008/10/23 | 1600       | 3      |
| 3    | 2008/09/02 | 700        | 1      |
| 4    | 2008/09/03 | 300        | 1      |
| 5    | 2008/08/30 | 2000       | NULL   |

- Apply inner Join on above tables and show the result
- Apply Left Join (or left outer join) on above tables and show the result.
- Apply Right Join (or right outer join) on above tables and show the result.
- Apply Full Join (or Full outer join) on above tables and show the result.

## Assignment:

**Deadline:** 27<sup>th</sup> Nov 2023

1. Search for the following commands in SQL server:

- Substring
- Ltrim
- Charindex
- Rtrim
- Replace
- Concatenating two columns
- Add constraint
- Over
- Convert

Also write SQL query for each of the above.

2. Write SQL Queries for the following:

- Write an SQL Query to copy a complete column from one table to another table's column by updating the previous records as well as inserting new records by giving an example with sample tables.
- Write an SQL Query to copy a complete table to another table by giving an example with sample tables.
- Write an SQL Query to copy data from one column to another column in the same table by giving an example with sample tables.